

Overall app idea

This is an app that lets parents track “Points” for their children, then the children will “cash in” their points for a reward. Points can be added or subtracted from each child, or can be zeroed out (when rewards are cashed in). The main goal of this app is give parents an authenticated experience to add/remove points for their children, and an unauthenticated unique URL for the children to track their points (but no ability to add/remove). The “reward” experience is not tracked in this app, it is simply tracking points, and when rewards are cashed in outside of the app, the parent has the ability to zero out points

Technical requirements

- This app should use modern authentication and authorization best practices (O/Auth2).
- When built locally, there should be a mock experience where the user can login with the username “admin” and the password “password”.
- This app is build using deno and will be deployed with deno deploy
- The favicon is included in the repo, but needs to be higher resolution.

Onboarding and login experience

1. Landing page.
 - Users should land on a login page when opening or navigating to this app.
 - There should be input fields for email and password.
 - There should be a login button that is disabled until the username and password fields are filled in
 - i. When the login button is pressed, it will authenticate the user and log them into the app.
 - ii. Only registered users should be able to login. All users not tracked in our database will fail.
 - There should be a “forgot password” link that takes you to the forgot password flow (described below)
 - There should be a “create account” button that takes you through an onboarding flow (described below)
2. Onboarding flow
 - When the user clicks the “Create account” button on the landing page, they are taken to the onboarding flow.
 - The onboarding flow will ask the following
 - i. First and last name
 - ii. Email address
 - iii. Password
 1. The password needs to be 12 characters long
 - iv. The option to add an authorized user
 1. This should be a + icon that, when clicked, shows an input field for another email address.

- v. When the user clicks submit, we need to store their data in a database. Deno has built-in options for databases. Make a decision on what the most robust, production-ready database option is (this will be a low traffic app, but I want to ensure it's secure).
 - 1. Any authenticated user should also get an email that takes them to the same onboarding flow described in this step.
 - a. They do not have the option to add an authorized user.
 - b. After they create an account, they now share admin responsibilities with the primary user
- 3. Forgot password flow
 - o From the landing page, if the user clicks forgot password, they are taken to the forgot password page.
 - o There is an email address input field
 - o There is a "recover password" button. It is disabled until the user enters a valid email address
 - i. A valid email address should be validated on the front end using a regex expression like


```
^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$
```

 or similar (based on your research)
 - o When the user clicks the recover password button, they are sent an email with a unique link to the "create new password" page.
- 4. Create new password page
 - o The user has clicked on the unique link that was generated from the forgot password flow
 - o They land on a page with 2 input fields: New password and confirm new password.
 - i. The passwords must match
 - ii. The password cannot be any of the user's last 3 passwords
 - iii. The password must be 12 characters long.
 - o There is a "reset password" button that is disabled until the above criteria are met
 - o When the user clicks the reset password button, update the password associated with their account and store it securely in our database.

Authenticated experience

- 1. First-time login
 - o The first time a user logs in, they land on a page that says "Welcome to Pointy!" with the logo stored in the src code repo called "pointy_logo.jpg"
 - o Below the welcome message, there is a "Add a child" section
 - i. It includes Child name which is required
 - ii. It also has the option to upload an image/icon. There should be some reasonable size restrictions on the image size.
 - o There is a "+ add another child" button/icon
 - i. When clicked, it populates another "add a child" section with the name input and ability to add an icon/image.

- ii. There is also a trash can icon that lets you remove the “add a child” section in case it was accidentally clicked
 - There is a “Save” button at the bottom right of the page.
 - i. It is disabled if any of the child name fields are empty
 - When the user clicks save, store the child info in the database associated with the account.
 - After a successful save, the user is taken to the “authenticated home page”
2. Authenticated home page
- The user must be logged in and have gone through the first time login experience in order to land on this page (they must have at least 1 child onboarded)
 - This home page should have a pointy logo from called “pointy_logo.jpg” on the top with the words “Pointy” next to it.
 - Under the pointy logo/header, there should be a list of all the children associated with the account with their icon/image displayed to the left of their name.
 - i. The images/icons should be a standard size so that everything looks uniform
 - Next to each child, put the following controls
 - i. A plus button that adds one point for the child



- 1. Use this as inspiration
- ii. A minus button that removes one point for the child



- 1. Use this as inspiration
- iii. A “Cash in points” button that resets the child’s point count to 0
- iv. There is also an edit icon next to each child.
 - 1. When the edit icon is clicked, the associated child’s info is editable.
 - a. The name field is prefilled with the existing child’s name and is editable.
 - b. The user has the ability to add a new icon/image
 - c. A save button populates and when clicked, the updated info is saved to the database.
 - i. This is only clickable if the name input is dirty or if a new image/icon is uploaded.
- The points data needs to be associated with each child and needs to persist in our stateful database
- At the very bottom of the page, under all the child names/icons, there should be a “+ add another child” button that allows the user to add another child.
 - i. When the user clicks the “+ add another child” button, the following appears

1. it populates another “add a child” section with the name input and ability to add an icon/image.
2. There is also a trash can icon that lets you remove the “add a child” section in case it was accidentally clicked
3. A save button that is disabled by default. It is enabled when the child's name is added.

Authenticated Point tracker page

1. This is a page where the child can track their points
2. This page needs to be a read-only page where the children can see their points.
3. It should look similar to Authenticated home page, without the controls to add/remove/cash in points.
 - a. Display the child's name and icons/images and the pointy header
4. This page should update in real time when points are added/removed for the child.
5. When a point is added, there should be a sweet css animation where stars explode from the existing number and the new number appears.
 - a. This should be accompanied by the attached “success_sound.mp3”
6. When a point is removed, there is no animation and the “fail_sound.mp3” should be played.